

nagi



OWNER & ADMIN GUIDE

Understand your own system.

A plain-language tour of everything you now own: how the app works by itself, how it is built, how it is sold, where the money and the keys flow, what is safe and what is not yet, and where it can grow. No technical background assumed.

Isha — version 0.5.0

Owner & Admin Guide — July 2026

Read once end to end; keep it beside the ROADMAP for the deep detail.

How to read this

You do not need to code to run Isha as a business. You do need to understand the shape of what you own. That is what this document gives you.

Everything here is in ordinary words. Where a real file or term matters, it appears in **bold** so you can search for it later, but you are never expected to open it. When something is a job only you can do, it is marked clearly. The companion **ROADMAP.md** holds the exact technical steps; this guide holds the understanding.

What you actually own

Isha is not one thing. It is three, and it helps to keep them separate in your head:

The piece	In one line	Where it lives
The app	The program customers install and use on Windows	Their PCs
The website	The page that explains Isha and takes payment	A web host you choose
The delivery path	Turns a payment into a working licence key	A payment provider (+ optionally one tiny server)

The rest of this guide walks through each, then covers safety, the rough edges, and where it can all go next.

How the app thinks (one brain, many mouths)

Isha has one decision-making core. Everything that can give it an instruction feeds that same core, so behaviour never changes depending on how the instruction arrived.

Imagine a single skilled assistant sitting at a desk. People can reach them four ways: by typing a note, by clicking a menu, by speaking, or by a standing rule ("every weekday at 9, do this"). No matter which way a request arrives, it lands on the **same desk** and is handled the same way. That is the single most important idea in how Isha is built.

What happens to one instruction, step by step

When someone says "open chrome", the request passes down a short assembly line. Each station does one small job:

- **Clean it up.** The raw words are tidied into simple pieces (lowercased, trimmed). This is also why a licence key uses only safe characters — so typing it can never get mangled.
- **Split it.** "mute and open chrome" becomes two separate requests.
- **Understand it.** Each request is read into a clear intention: the action (open) and the target (chrome).
- **Check and resolve it.** Isha works out what "chrome" really means on this PC, and flags anything risky as "needs a yes first".
- **Do it.** The action runs, and a snapshot is saved first so it can be undone.
- **Report plainly.** One honest sentence comes back: what happened, or why it couldn't. Never a fake success.

Why this matters to you

Because there is only one core, a fix or a new capability is added in one place and instantly works everywhere — typed, clicked, spoken, or automatic. It is what keeps a small app dependable as it grows, and it is why the app is cheap to maintain.

Where a customer's settings live

Everything a customer sets up — their modes, their hotkey, their aliases — is kept in a plain **settings file** in their own user folder. It is simple on purpose: easy to back up, easy to inspect, and it never leaves their machine. The one sensitive item, their licence (name and email), is stored **encrypted to their Windows account**, not in plain view.

The pieces, plainly

You will see many small files with names beginning "a_". Think of each as one specialised tool the core can pick up. They are deliberately small and independent — if one is missing or unavailable, it steps aside and the rest keep working. Grouped by what they do:

Group	What it handles
The core pipeline	Reading and carrying out instructions (the assembly line above)
Modes	Creating, activating and closing groups of apps and websites
Auto-triggers	The background watcher that fires modes on time, battery, idle, or an app opening
System actions	Volume, mute, lock, shutdown, sleep — the direct controls
Utilities	Screenshots, disk space, internet check, clipboard history, the focus timer
The face (palette, tray, settings)	The small windows a customer sees and the tray icon
Licensing	Checking a licence key, entirely offline
Updates	Checking for and installing a new version safely
Issue reporting	Building a local problem report the customer can choose to send

What runs by itself

A few small helpers stay quietly awake the whole time Isha is running. They are what make it feel effortless.

None of these ever act drastically on their own. They watch, and at most they open or close the apps you have already grouped into a mode.

Quiet helper	What it is always doing	Cost
Hotkey listener	Waiting for the keystroke that opens the bar	Effectively none
Tray icon	Showing Isha is alive; offering its menu	Effectively none
Mode watcher	Checking every 20 seconds whether an auto-trigger should fire	Tiny
Clipboard watcher	Remembering recent things copied, for clipboard history	Tiny

A deliberate safety line in the automatic parts

The mode watcher can open and close your app groups automatically, but it is built so it can NEVER run a mode's free-text "script" on its own — that always needs a live human yes. An unattended part of the system is never allowed to run an arbitrary command. Remember this one sentence; it is the heart of Isha's safety design.

How selling works

The guiding rule is: keep the backend as small as possible. Ideally, zero servers to start.

The model: a Merchant of Record

You sell Isha once, for a single price, through a **Merchant of Record** (the recommended ones are **Lemon Squeezy** or **Paddle**). A Merchant of Record is not just a card processor — they become the legal seller. They take the payment, charge the correct sales tax (VAT/GST) for wherever the buyer is, and hand it to the tax authorities. You never register for tax in other countries and never touch a card number. This is the single biggest headache they remove for you.

The licence: yours, and offline

Even though the payment company handles the money, the **licence key stays yours**. It is a small signed token that the app checks entirely on the customer's own machine — no sign-in, no phone-home. That keeps Isha's core promise ("nothing leaves your machine") intact even for activation. You keep only the buyer's name and email, for delivery.

From payment to working app, in one picture

1. A customer pays on your website through the Merchant of Record.
2. That triggers a licence key to be created and signed with **your private key** (the secret that proves a key is genuine).
3. The key is emailed to the customer.
4. They type **activate license <key>** into Isha, and it verifies on the spot, offline.

Start manual, automate later

You do not need any server on day one:

Stage	What you do	When
Manual	When a sale email arrives, run one command to make the key and reply with it	Your first sales
Automatic	A tiny server (the "webhook") makes and emails the key the instant a sale completes	When manual gets tedious

Both produce the exact same kind of key. The automatic path was already built and tested; you switch to it whenever you like.

The one secret you must protect

The **private signing key** is what mints valid licences. Anyone who has it can create licences for free. It must be generated by you, kept offline (a password manager or hardware key), never emailed, never put inside the app, and never uploaded anywhere except as a protected secret on your automatic server if you run one. Losing control of it is the only truly expensive mistake in the whole system.

How you ship a new version

Turning the source code into the single file customers install is a Windows-only job, done with scripts that are already written. In short:

1. **Build.** One command packages the code into an app folder and wraps it into a single **IshaSetup** installer.
2. **Test.** You run it on a real Windows PC and confirm the tray icon, hotkey, and a few commands work.
3. **Sign.** With a code-signing certificate (a paid yearly item), the installer is stamped as genuinely from you, which calms Windows' first-run warning.
4. **Publish.** You upload the installer to a download location and point the purchase email at it.

Two paid, one-time-ish setup items

A code-signing certificate (roughly a few hundred dollars a year) and a domain name (about ten dollars a year) are the only real costs to get selling. The payment provider takes a small cut per sale instead of a fixed fee. Everything else — hosting the website, the build tools — has a free tier.

Security, honestly

Isha is careful about the things it genuinely can control, and honest about the things no desktop app can. Both halves matter.

The line Isha defends

Isha protects the boundary between **this Windows user account** and everything outside it: another account on the same PC, a planted or tampered file, a hostile network, or a faked update. Within that line, several specific risks were found and closed:

The risk (in plain terms)	How it is handled now
A tampered list of app locations could make "open chrome" launch a fake	The location is re-checked at launch; the list is treated as a hint, never an authority
An automatic trigger could silently run a hidden command	Automatic parts can never run a mode script; it always needs a live yes
A downloaded update could be swapped for a bad file before it runs	The update is verified again the instant before it launches, in a private folder
The customer's email sat in plain view in a settings file	It is now encrypted to their Windows account
A faked update could pretend to be genuine	Updates must carry your cryptographic signature or they are refused outright

The line Isha does not pretend to defend

If a PC is **already infected** — malware running as that same user, or a keylogger already installed — no desktop program, in any product, can fully protect against it. Once hostile code runs as you, it can do anything you can. Isha states this plainly rather than advertising protection it cannot deliver. Being honest here is itself a security feature: it sets correct expectations.

Why the licence is a soft gate

An unlicensed copy of Isha still works. This is deliberate. At this price, aggressive anti-piracy would mostly punish honest customers with lockouts and bugs. The licence supports the project and is checked gently, once, without ever blocking features. The security work around licences is about protecting the buyer's personal data, not about locking anyone out.

Known rough edges and things to watch

An honest owner's list. None of these are hidden faults — they are the true state of the system, so you are never surprised.

Rough edge	What it means for you	What to do
Placeholder keys ship by default	Until you generate real keys, every licence and update correctly fails	Generate them once, offline, before your first sale
Not code-signed yet	Windows shows a first-run caution to buyers	Buy a certificate; an EV one clears it instantly
Windows code not yet run on Windows	The app was developed on Linux; its Windows-only parts need a real first run	Do the build-and-test pass on a Windows PC (the biggest open task)
The visible windows use one UI thread	A slow action could briefly freeze the small window	Fine at today's scale; the planned "Nagi" UI rework fixes it properly
The mode watcher polls every 20s	A negligible but non-zero background cost	Acceptable; can be tuned later
No usage analytics by design	You cannot see crashes unless a customer reports one	Keep the report feature friendly; consider opt-in analytics later
Manual fulfilment does not scale	Copy-pasting keys is fine for a trickle, not a flood	Switch on the automatic webhook when volume grows
Voice needs an add-on	Voice is not in the base install	Document it; bundle it later if demand is real

Where it can grow

Isha is intentionally small today. Its shape — one core, many small tools — makes it easy to extend without rewrites.

Near-term, high value

- **The "Nagi" calm UI.** A designed, breathing interface (already fully specified) replacing the plain windows — a big jump in polish, buildable one small step at a time.
- **More auto-triggers.** Location, calendar events, headphones connected, and similar conditions, using the same watcher.
- **A friendlier settings screen** for modes, hotkeys and triggers, so power users never touch the settings file.

Business growth

- **Automatic delivery** (the webhook) once sales pick up, and later **team/multi-seat licences** for offices.
- **A signed auto-update feed** so existing customers update in place — smoother releases and easier fixes.
- **A localised website and app** (Hinglish is already supported in commands; the marketing can follow).
- **India-specific payments** (Razorpay) once volume there justifies registering a local entity — a Merchant of Record covers India legally until then.

Bigger bets

- **Optional, opt-in cloud sync** of modes across a customer's own devices — kept strictly opt-in to preserve the privacy promise.
- **A plugin system** so modes can drive more kinds of actions safely.
- **Other platforms** (macOS, Linux) reusing the same core; only the system-touching tools need porting.

Your responsibilities, on one page

The short list of things only you can own. Do these and the system runs cleanly.

- **Generate the real keys once, offline**, and paste the public halves into the app before your first release.
- **Guard the private signing keys forever** — offline, backed up, never committed, never emailed.
- **Do the Windows build-and-test pass** before selling, and again for every new version.
- **Buy and keep current** a code-signing certificate and your domain name.
- **Open a Merchant-of-Record account**, create the product, and put the checkout link on the website.
- **Publish the website** with its privacy, terms, and refund pages filled in (the payment provider requires them).
- **Deliver keys** — by hand at first, via the webhook once it is worth automating.
- **Keep a friendly support inbox** — with no analytics, customer reports are your eyes.

Plain-English glossary

Term	What it really means
Merchant of Record	A company that becomes the legal seller and handles all sales tax and card data for you
Licence key	A short signed code proving someone bought Isha; checked offline on their PC
Private / public key	A matched pair: the private one (secret, yours) creates licences; the public one (inside the app) checks them
Webhook	A tiny automatic notice the payment company sends your server the instant a sale completes
Code signing	Stamping your installer as genuinely from you, so Windows trusts it
SmartScreen	Windows' first-run caution for unknown apps; signing and reputation calm it
DPAPI	Windows' built-in way to encrypt data so only that user account can read it
Soft gate	A licence check that informs but never blocks features
Auto-trigger	A standing rule that activates a mode on a condition (time, battery, idle, app launch)
Mode	A named group of apps and websites opened and closed together — Isha's headline feature